BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 4

Roll no: 42 PRN No: 12414728 Name of Student: **Saachi Laturiya**

Subject Name : Deep Learning

Problem Statement : Develop an LSTM-based model for time-series forecasting using a sales dataset and evaluate its performance using appropriate evaluation metrics.

Algorithm :

1. Import the required libraries for data processing, visualization, and deep learning.
2. Generate a monthly sales dataset containing trend, seasonal variation, and random noise.
3. Normalize the sales data using MinMaxScaler.
4. Split the dataset into training and testing sets.
5. Create sequences of 12 consecutive months to predict the next month's sales.
6. Build an LSTM model with 50 units, a dropout layer, and a dense output layer.
7. Compile the model using the Adam optimizer and mean squared error loss.
8. Train the model for 30 epochs using the training dataset.
9. Predict sales values for the test dataset and convert them back to the original scale.
10. Evaluate the model using MAE, RMSE, and MAPE.
11. Plot actual versus predicted sales and visualize training and validation loss.

Code :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
```

```python
# Generate monthly sales data
np.random.seed(42)

months = pd.date_range(start="2015-01-01", periods=120, freq="MS")

trend = np.linspace(100, 300, 120)
seasonality = 30 * np.sin(2 * np.pi * np.arange(120) / 12)
noise = np.random.normal(0, 10, 120)

sales = trend + seasonality + noise

df = pd.DataFrame({
    "Date": months,
    "Sales": sales
})

print(df.head())
print("\nDataset shape:", df.shape)
```

```
        Date       Sales
0 2015-01-01  104.967142
1 2015-02-01  115.298029
2 2015-03-01  135.818992
3 2015-04-01  150.272315
4 2015-05-01  130.361917

Dataset shape: (120, 2)
```

```python
# Normalize sales data
scaler = MinMaxScaler(feature_range=(0, 1))

sales_values = df[["Sales"]].values

# Split data into training and testing sets (80:20)
split = int(len(sales_values) * 0.8)

train_data = sales_values[:split]
test_data = sales_values[split:]

# Fit scaler only on training data
train_scaled = scaler.fit_transform(train_data)
test_scaled = scaler.transform(test_data)

# Create sequences
def create_sequences(data, lookback=12):
    X, y = [], []

    for i in range(lookback, len(data)):
        X.append(data[i-lookback:i, 0])
        y.append(data[i, 0])

    return np.array(X), np.array(y)

# Training sequences
X_train, y_train = create_sequences(train_scaled)
```

```python
# Testing sequences (include previous training observations)
combined_data = np.concatenate(
    (train_scaled[-12:], test_scaled), axis=0
)

X_test, y_test = create_sequences(combined_data)

# Reshape for LSTM input: samples, timesteps, features
X_train = X_train.reshape(X_train.shape[0], 12, 1)
X_test = X_test.reshape(X_test.shape[0], 12, 1)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```python
# Build LSTM model
model = Sequential([
    LSTM(50, activation="tanh", input_shape=(12, 1)),
    Dropout(0.2),
    Dense(1)
])

# Compile model
model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

# Display model architecture
model.summary()

# Train model
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_split=0.1,
    verbose=1
)
```

```python
# Generate predictions
y_pred = model.predict(X_test)

# Convert values back to original sales scale
y_pred_actual = scaler.inverse_transform(y_pred)
y_test_actual = scaler.inverse_transform(
    y_test.reshape(-1, 1)
)

# Calculate evaluation metrics
mae = mean_absolute_error(y_test_actual, y_pred_actual)
rmse = np.sqrt(mean_squared_error(y_test_actual, y_pred_actual))
mape = np.mean(
    np.abs((y_test_actual - y_pred_actual) / y_test_actual)
) * 100

print("Model Evaluation:")
print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"MAPE: {mape:.2f}%")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 158ms/step
Model Evaluation:
MAE: 22.02
RMSE: 26.65
MAPE: 7.62%
```

```python
# Plot actual and predicted sales
plt.figure(figsize=(12, 5))

plt.plot(
    df["Date"].iloc[split:],
    y_test_actual.flatten(),
    label="Actual Sales"
)

plt.plot(
    df["Date"].iloc[split:],
    y_pred_actual.flatten(),
    label="Predicted Sales"
)

plt.title("Actual vs Predicted Sales Using LSTM")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("LSTM Model Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.grid(True)
plt.show()
```

Output :

Figure 1: Actual vs. Predicted Sales Using LSTM

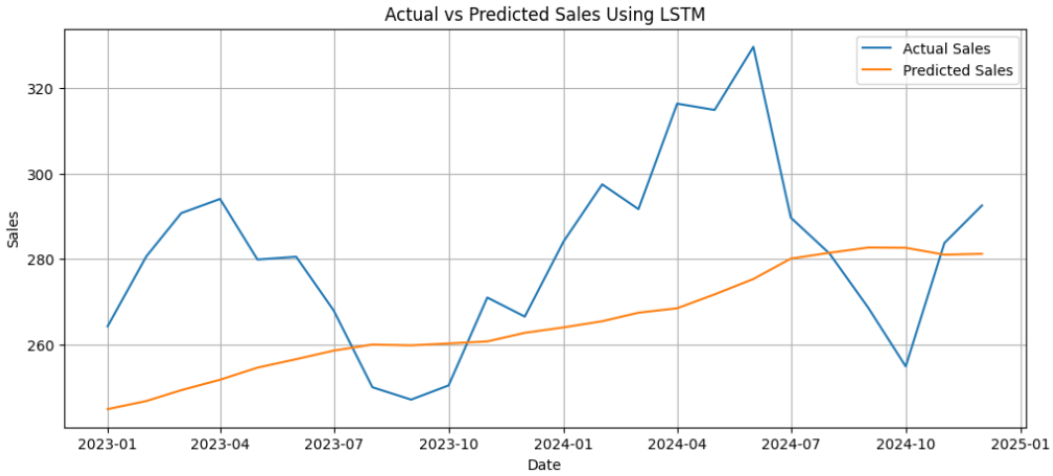
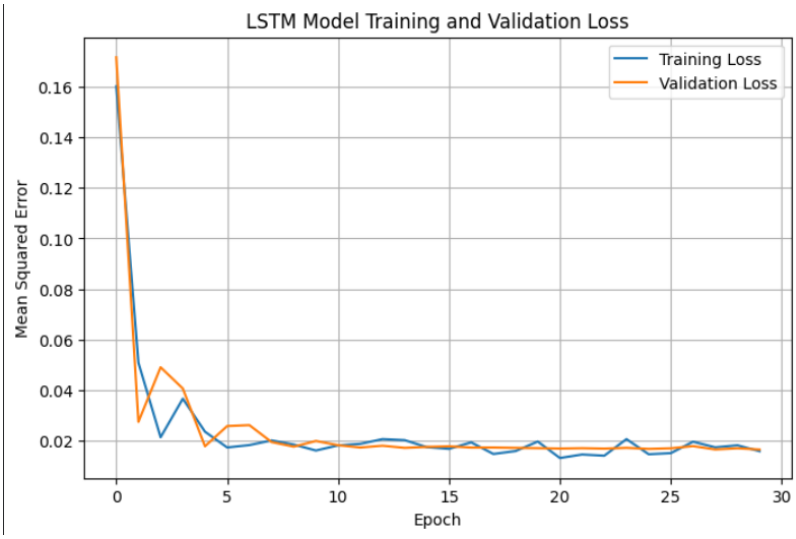


Figure 2: LSTM Model Training and Validation Loss



Conclusion : An LSTM-based model was developed for monthly sales forecasting using a synthetically generated dataset. The model was trained using historical sales sequences and evaluated using MAE, RMSE, and MAPE. The actual and predicted sales were visualized to analyze forecasting performance, while training and validation loss were used to monitor the model's learning behavior.